

Practical: - 1

Aim: - Write a program to sort given elements of an array in ascending order using bubble sort. Analyse the time complexity for best, average and worst case.

➤ Bubble Sort Algorithm

Bubble Sort is a simple comparison-based sorting algorithm. It repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The process is repeated until the list is sorted.

Step 1: Start

Step 2: Repeat the following for $i = 0$ to $n-1$

 Set swapped = false

 For $j = 0$ to $n-i-2$

 If $arr[j] > arr[j+1]$

 Swap $arr[j]$ and $arr[j+1]$

 Set swapped = true

 If swapped = false

 Break the loop

Step 3: Stop

➤ Bubble Sort Code: -

```
public class BubbleSort{
    // Bubble sort function to sort array in ascending order
    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        boolean swapped;
        // Outer loop for each element
        for (int i = 0; i < n - 1; i++) {
            swapped = false;
            // Inner loop for comparing adjacent elements
            for (int j = 0; j < n - i - 1; j++) {
                // Swap if left element is greater than right element
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swapped = true; // A swap occurred
                }
            }
            // If no swaps occurred in this pass, array is already sorted
            if (!swapped) {
                break;
            }
        }
    }
    // Function to print the array
    public static void printArray(int[] arr) {
        System.out.print("[");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i]);
            if (i < arr.length - 1) {
                System.out.print(", ");
            }
        }
        System.out.print("]");
    }
}
```

```
    }  
    }  
    System.out.println("]");  
}  
  
public static void main(String[] args) {  
  
    int[] data = {64, 25, 12, 22, 11};  
    // Print original array  
    System.out.print("Original array: ");  
    printArray(data);  
    // Record the start time before sorting  
    long startTime = System.nanoTime();  
    // Perform bubble sort  
    bubbleSort(data);  
    // Record the end time after sorting  
    long endTime = System.nanoTime();  
    // Calculate execution time in nanoseconds  
    long executionTime = endTime - startTime;  
    // Print the sorted array  
    System.out.print("Sorted array:  ");  
    printArray(data);  
    // Print execution time  
    System.out.println("Execution Time: " + executionTime + " nanoseconds");  
}  
}
```

➤ **Output: -**

```
PS D:\RISHI\ADIT-sem_5\DAA> javac BubbleSort.java  
PS D:\RISHI\ADIT-sem_5\DAA> java BubbleSort  
Original array: [64, 25, 12, 22, 11]  
Sorted array:   [11, 12, 22, 25, 64]  
Execution Time: 6400 nanoseconds  
PS D:\RISHI\ADIT-sem_5\DAA> java BubbleSort  
Original array: [64, 25, 12, 22, 11]  
Sorted array:   [11, 12, 22, 25, 64]  
Execution Time: 5000 nanoseconds  
PS D:\RISHI\ADIT-sem_5\DAA> javac BubbleSort.java  
PS D:\RISHI\ADIT-sem_5\DAA> java BubbleSort  
Original array: [64, 25, 12, 22, 11]  
Sorted array:   [11, 12, 22, 25, 64]  
Execution Time: 6900 nanoseconds
```

➤ **Conclusion: -**

Bubble sort is easy to implement but inefficient to large datasets. It is suitable for small arrays or when the array is almost sorted. Also we get that the worst case time complexity of Bubble Sort is $O(n^2)$.

Practical: - 2

Aim: - Write a program to sort given elements of an array in ascending order using selection sort. Analyse the time complexity for best, average and worst case.

➤ Selection Sort Algorithm

Selection Sort is a comparison-based sorting algorithm. It works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the array and swapping it with the first unsorted element.

Step 1: Start

Step 2: Repeat for $i = 0$ to $n-2$

 Set $\text{min_index} = i$

 For $j = i+1$ to $n-1$

 If $\text{arr}[j] < \text{arr}[\text{min_index}]$

 Set $\text{min_index} = j$

 Swap $\text{arr}[i]$ and $\text{arr}[\text{min_index}]$

Step 3: Stop

➤ Selection Sort Code: -

```
public class SelectionSort {
    // Function to perform selection sort
    public static void selectionSort(int[] arr) {
        int n = arr.length;
        // Outer loop for each element
        for (int i = 0; i < n - 1; i++) {
            int minIndex = i;
            // Find the minimum element in unsorted array
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[minIndex]) {
                    minIndex = j;
                }
            }
            // Swap the found minimum element with the first element
            int temp = arr[minIndex];
            arr[minIndex] = arr[i];
            arr[i] = temp;
        }
    }
    // Function to print the array
    public static void printArray(int[] arr) {
        System.out.print("[");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i]);
            if (i < arr.length - 1) {
                System.out.print(", ");
            }
        }
        System.out.println("]");
    }
    public static void main(String[] args) {
        // Define a sample array
        int[] data = {29, 10, 14, 37, 13, 38, 20, 30, 29};

        // Print original array
```

```
        System.out.print("Original array: ");
        printArray(data);
        // Record the start time before sorting
        long startTime = System.nanoTime();
        // Perform selection sort
        selectionSort(data);
        // Record the end time after sorting
        long endTime = System.nanoTime();
        // Calculate execution time in nanoseconds
        long executionTime = endTime - startTime;
        // Print the sorted array
        System.out.print("Sorted array:  ");
        printArray(data);
        // Print execution time
        System.out.println("Execution Time: " + executionTime + " nanoseconds");
    }
}
```

➤ **Output: -**

```
PS D:\RISHI\ADIT-sem_5\DAA> javac SelectionSort.java
PS D:\RISHI\ADIT-sem_5\DAA> java SelectionSort
Original array: [29, 10, 14, 37, 13]
Sorted array:   [10, 13, 14, 29, 37]
Execution Time: 6200 nanoseconds
PS D:\RISHI\ADIT-sem_5\DAA> javac SelectionSort.java
PS D:\RISHI\ADIT-sem_5\DAA> java SelectionSort
Original array: [29, 10, 14, 37, 13, 38, 20, 30, 29]
Sorted array:   [10, 13, 14, 20, 29, 29, 30, 37, 38]
Execution Time: 8700 nanoseconds
PS D:\RISHI\ADIT-sem_5\DAA> █
```

➤ **Conclusion: -**

The Selection Sort algorithm was successfully implemented in Java. It performs well on small datasets but is inefficient for larger arrays due to its quadratic time complexity. It is, however, better than bubble sort in terms of number of swaps.

Practical: - 3

Aim: - Write a program to implement heap sort.

➤ What is Heap Sort?

- Heap Sort is a comparison-based sorting technique that uses a binary heap data structure. A binary heap is a complete binary tree which satisfies the heap property:
- In a Max Heap, for any node 'i', the value at 'i' is greater than or equal to the values of its children.
- In a Min Heap, for any node 'i', the value at 'i' is less than or equal to the values of its children.
- Heap Sort typically uses a Max Heap to sort elements in ascending order.
- Heap Sort is NOT stable. It may change the order of equal elements because of arbitrary swaps during heapify.

➤ Working of Heap Sort

- Heap Sort consists of two main phases:
 - o Build a Max Heap from the unsorted input array.
 - o Repeatedly extract the maximum element from the heap and move it to the end of the array. Reduce the heap size and heapify the root.

➤ Step-by-step Process:

1. Build Max Heap:
 - a. Convert the array into a max heap by calling 'heapify' from the last non-leaf node to the root.
 - b. For an array of size 'n', the last non-leaf node is at index $(n / 2) - 1$.
2. Sorting:
 - a. Swap the first (largest) element with the last.
 - b. Reduce the heap size by one.
 - c. Call 'heapify' on the root to maintain the heap property.
 - d. Repeat until the heap size is 1.

➤ Heap Sort Code: -

```
public class HeapSort{
    // Function to sort an array using heap sort
    public static void heapSort(int[] arr) {
        int n = arr.length;
        // Step 1: Build max heap
        for (int i = n / 2 - 1; i >= 0; i--) {
            heapify(arr, n, i);
        }
        // Step 2: Extract elements from heap one by one
        for (int i = n - 1; i > 0; i--) {
            // Move current root (largest) to end
            int temp = arr[0];
            arr[0] = arr[i];
            arr[i] = temp;

            // Call max heapify on the reduced heap
            heapify(arr, i, 0);
        }
    }
    // Function to maintain heap property (max-heap)
    public static void heapify(int[] arr, int heapSize, int rootIndex) {
        int largest = rootIndex;
        int left = 2 * rootIndex + 1; // Left child
        int right = 2 * rootIndex + 2; // Right child
        // If left child is larger than root
        if (left < heapSize && arr[left] > arr[largest]) {
```

```

        largest = left;
    }

    // If right child is larger than current largest
    if (right < heapSize && arr[right] > arr[largest]) {
        largest = right;
    }
    // If largest is not root, swap and heapify again
    if (largest != rootIndex) {
        int swap = arr[rootIndex];
        arr[rootIndex] = arr[largest];
        arr[largest] = swap;

        // Recursively heapify the affected subtree
        heapify(arr, heapSize, largest);
    }
}
// Utility function to print an array
public static void printArray(int[] arr) {
    System.out.print("[");
    for (int i = 0; i < arr.length; i++) {
        System.out.print(arr[i]);
        if (i < arr.length - 1)
            System.out.print(", ");
    }
    System.out.println("]");
}
// Main method
public static void main(String[] args) {
    int[] data = {20, 50, 30, 10, 60, 90, 70, 40, 80, 100};
    System.out.print("Original array: ");
    printArray(data);
    long startTime = System.nanoTime();
    heapSort(data);
    long endTime = System.nanoTime();
    long executionTime = endTime - startTime;
    System.out.print("Sorted array: ");
    printArray(data);
    System.out.println("Execution Time: " + executionTime + " nanoseconds");
}
}

```

➤ Output: -

```

PS D:\RISHI\ADIT-sem_5\DAA> javac HeapSort.java
PS D:\RISHI\ADIT-sem_5\DAA> java HeapSort
PS D:\RISHI\ADIT-sem_5\DAA> java HeapSort
Original array: [20, 50, 30, 10, 60, 90, 70]
Sorted array:   [10, 20, 30, 50, 60, 70, 90]
Execution Time: 12500 nanoseconds
PS D:\RISHI\ADIT-sem_5\DAA> javac HeapSort.java
PS D:\RISHI\ADIT-sem_5\DAA> java HeapSort
Original array: [20, 50, 30, 10, 60, 90, 70, 40, 80, 100]
Sorted array:   [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
Execution Time: 17400 nanoseconds
PS D:\RISHI\ADIT-sem_5\DAA> 

```

➤ Conclusion: -

Heap Sort is a robust and efficient sorting algorithm with consistent performance across all scenarios. While it is not stable, it excels in environments with memory constraints due to its in-place nature. It is particularly useful when consistent time complexity is needed regardless of the input.

Practical: - 4

Aim: - Write a program to search given element from an array using sequential search and binary search. Analyse the time complexity for best, average and worst case.

➤ **Sequential Search: -**

In Sequential Search, each element in the array is checked one by one from the beginning until the target element is found or the end of the array is reached.

Step 1: Start

Step 2: Traverse the array from index 0 to n-1

Step 3: For each index i, if $arr[i] == key$, return i

Step 4: If not found, return -1

Step 5: Stop

➤ **Binary Search: -**

Binary Search works on a sorted array. It repeatedly divides the search interval in half:

- If the key is less than the middle element, search the left half.
- If the key is greater, search the right half.
- If equal, return the index.

Step 1: Start

Step 2: Set low = 0 and high = n-1

Step 3: While low <= high:

 mid = (low + high) / 2

 If $arr[mid] == key$, return mid

 If $arr[mid] > key$, high = mid - 1

 Else low = mid + 1

Step 4: If not found, return -1

Step 5: Stop

➤ **Code: -**

```
import java.util.Arrays;
import java.util.Scanner;

public class SequentialBinarySearch{

    // Linear/Sequential Search
    public static int linearSearch(int[] arr, int key) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == key)
                return i;
        }
        return -1;
    }

    // Binary Search (on sorted array)
    public static int binarySearch(int[] arr, int key) {
        int low = 0;
        int high = arr.length - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (arr[mid] == key)
```

```
        return mid;
    else if (arr[mid] < key)
        low = mid + 1;
    else
        high = mid - 1;
    }

    return -1;
}

// Utility function to print array
public static void printArray(int[] arr) {
    System.out.print("[");
    for (int i = 0; i < arr.length; i++) {
        System.out.print(arr[i]);
        if (i < arr.length - 1)
            System.out.print(", ");
    }
    System.out.println("]");
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    // Sample array
    int[] data = {23, 5, 87, 12, 19, 44, 31};
    int[] sortedData = Arrays.copyOf(data, data.length);
    Arrays.sort(sortedData); // Required for binary search

    System.out.print("Array: ");
    printArray(data);

    System.out.print("Enter element to search: ");
    int key = sc.nextInt();

    // Linear Search
    long startLinear = System.nanoTime();
    int linearIndex = linearSearch(data, key);
    long endLinear = System.nanoTime();

    // Binary Search
    long startBinary = System.nanoTime();
    int binaryIndex = binarySearch(sortedData, key);
    long endBinary = System.nanoTime();

    // Results
    System.out.println("\n--- Linear Search ---");
    if (linearIndex != -1)
        System.out.println("Element found at index: " + linearIndex);
    else
        System.out.println("Element not found.");
    System.out.println("Execution Time: " + (endLinear - startLinear) + " ns");

    System.out.println("\n--- Binary Search (Sorted Array) ---");
    if (binaryIndex != -1)
```



```
        System.out.println("Element found at index (sorted array): " +  
        binaryIndex);  
    }  
    else  
    {  
        System.out.println("Element not found.");  
        System.out.println("Execution Time: " + (endBinary - startBinary) + " ns");  
        sc.close();  
    }  
}
```

➤ **Output: -**

```
PS D:\RISHI\ADIT-sem_5\DAA> javac SequentialBinarySearch.java  
PS D:\RISHI\ADIT-sem_5\DAA> java SequentialBinarySearch  
Array: [23, 5, 87, 12, 19, 44, 31]  
Enter element to search: 19  
  
--- Linear Search ---  
Element found at index: 4  
Execution Time: 2500 ns  
  
--- Binary Search (Sorted Array) ---  
Element found at index (sorted array): 2  
Execution Time: 1200 ns  
PS D:\RISHI\ADIT-sem_5\DAA> java SequentialBinarySearch  
Array: [23, 5, 87, 12, 19, 44, 31]  
Enter element to search: 50  
  
--- Linear Search ---  
Element not found.  
Execution Time: 3600 ns  
  
--- Binary Search (Sorted Array) ---  
Element not found.  
Execution Time: 2800 ns  
PS D:\RISHI\ADIT-sem_5\DAA> █
```

➤ **Conclusion: -**

Linear Search is simple but inefficient for large arrays. Binary Search is significantly faster but only works on sorted arrays. In real-world applications, binary search is preferred when fast searching is needed and sorting is acceptable.

Practical: - 5

Aim: - Write a program to sort given elements of an array in ascending order using merge sort. Analyze the time complexity for best, average and worst case.

➤ **Merger Sort Algorithm: -**

- Merge Sort is a Divide and Conquer algorithm. It works by:
 - o Dividing the array into two halves.
 - o Recursively sorting both halves.
 - o Merging the sorted halves into one sorted array.
- Stable Sort, Not in-place (uses extra memory), Works efficiently on large datasets, Used in applications like external sorting and databases
- Algorithm (High-Level):
MergeSort(arr[], left, right):
 If left < right:
 mid = (left + right) / 2
 MergeSort(arr, left, mid)
 MergeSort(arr, mid + 1, right)
 Merge(arr, left, mid, right)

➤ **Merge Sort Code: -**

```
public class MergeSort{

    // Function to merge two subarrays of arr[]
    public static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;

        // Create temp arrays
        int[] leftArr = new int[n1];
        int[] rightArr = new int[n2];

        // Copy data to temp arrays
        for (int i = 0; i < n1; i++)
            leftArr[i] = arr[left + i];
        for (int j = 0; j < n2; j++)
            rightArr[j] = arr[mid + 1 + j];

        // Merge the temp arrays

        int i = 0, j = 0;
        int k = left;

        while (i < n1 && j < n2) {
            if (leftArr[i] <= rightArr[j]) {
                arr[k] = leftArr[i];
                i++;
            } else {
                arr[k] = rightArr[j];
                j++;
            }
            k++;
        }

        // Copy remaining elements
```

```
        while (i < n1) {
            arr[k] = leftArr[i];
            i++;
            k++;
        }

        while (j < n2) {
            arr[k] = rightArr[j];
            j++;
            k++;
        }
    }

    // Function to implement merge sort
    public static void mergeSort(int[] arr, int left, int right) {
        if (left < right) {
            int mid = (left + right) / 2;

            // Sort first and second halves
            mergeSort(arr, left, mid);
            mergeSort(arr, mid + 1, right);

            // Merge the sorted halves
            merge(arr, left, mid, right);
        }
    }

    // Utility function to print an array
    public static void printArray(int[] arr) {
        System.out.print("[");
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i]);
            if (i < arr.length - 1)
                System.out.print(", ");
        }
        System.out.println("]");
    }

    // Main method
    public static void main(String[] args) {
        int[] data = {38, 27, 43, 3, 9, 82, 10, 2, 30, 22, 50};

        System.out.print("Original array: ");
        printArray(data);

        long startTime = System.nanoTime();

        mergeSort(data, 0, data.length - 1);

        long endTime = System.nanoTime();
        long executionTime = endTime - startTime;

        System.out.print("Sorted array: ");
        printArray(data);

        System.out.println("Execution Time: " + executionTime + " nanoseconds");
    }
}
```

```
}  
}
```

➤ **Output: -**

```
PS D:\RISHI\ADIT-sem_5\DAA> javac MergeSort.java  
PS D:\RISHI\ADIT-sem_5\DAA> java MergeSort  
Original array: [38, 27, 43, 3, 9, 82, 10]  
Sorted array:   [3, 9, 10, 27, 38, 43, 82]  
Execution Time: 28800 nanoseconds  
PS D:\RISHI\ADIT-sem_5\DAA> java MergeSort  
Original array: [38, 27, 43, 3, 9, 82, 10]  
Sorted array:   [3, 9, 10, 27, 38, 43, 82]  
Sorted array:   [3, 9, 10, 27, 38, 43, 82]  
Execution Time: 28800 nanoseconds  
PS D:\RISHI\ADIT-sem_5\DAA> javac MergeSort.java  
PS D:\RISHI\ADIT-sem_5\DAA> java MergeSort  
Original array: [38, 27, 43, 3, 9, 82, 10, 2, 30, 22, 50]  
Sorted array:   [2, 3, 9, 10, 22, 27, 30, 38, 43, 50, 82]  
Execution Time: 13300 nanoseconds  
PS D:\RISHI\ADIT-sem_5\DAA> 
```

➤ **Conclusion: -**

Merge Sort was successfully implemented in Java. It provides consistent $O(n \log n)$ performance for all input types. It is suitable for sorting large datasets and is stable, but not in-place due to additional memory usage.

Practical: - 6

Aim: - Write a program to sort given elements of an array in ascending order using quick sort. Analyse the time complexity for best, average and worst case.

➤ **Quick Sort Algorithm: -**

Quick Sort is a Divide and Conquer algorithm. It selects a pivot element and partitions the array into two sub-arrays:

- Elements less than pivot go to the left
- Elements greater than pivot go to the right

The process is recursively applied to both partitions until the whole array is sorted.

How it works:

- Choose a pivot (typically the last element)
- Partition the array so that all elements less than pivot come before it and greater ones after it.
- Recursively apply the same steps to sub-arrays.

Algorithm (Quick Sort):

QuickSort(arr[], low, high):

if low < high:

 pi = partition(arr, low, high)
 QuickSort(arr, low, pi - 1)
 QuickSort(arr, pi + 1, high)

partition(arr[], low, high):

 pivot = arr[high]
 i = low - 1
 for j = low to high - 1:
 if arr[j] < pivot:
 i++
 swap arr[i] and arr[j]
 swap arr[i + 1] and arr[high]
 return i + 1

➤ **Quick Sort Code: -**

```
public class QuickSort{  
    // Function to perform partition  
    public static int partition(int[] arr, int low, int high) {  
        int pivot = arr[high]; // Choose last element as pivot  
        int i = low - 1; // Index of smaller element  
  
        for (int j = low; j < high; j++) {  
            if (arr[j] < pivot) {  
                i++;  
                // Swap arr[i] and arr[j]  
                int temp = arr[i];  
                arr[i] = arr[j];  
                arr[j] = temp;  
            }  
        }  
        // Swap arr[i + 1] and arr[high] (pivot)  
        int temp = arr[i + 1];  
        arr[i + 1] = arr[high];  
        arr[high] = temp;  
  
        return i + 1;  
    }  
}
```

```
}  
// Function to perform quick sort  
public static void quickSort(int[] arr, int low, int high) {  
    if (low < high) {  
        // Partitioning index  
        int pi = partition(arr, low, high);  
  
        // Recursively sort elements before and after partition  
        quickSort(arr, low, pi - 1);  
        quickSort(arr, pi + 1, high);  
    }  
}  
// Utility function to print an array  
public static void printArray(int[] arr) {  
    System.out.print("[");  
    for (int i = 0; i < arr.length; i++) {  
        System.out.print(arr[i]);  
        if (i < arr.length - 1)  
            System.out.print(", ");  
    }  
    System.out.println("]");  
}  
// Main method  
public static void main(String[] args) {  
    int[] data = {34, 7, 23, 32, 5, 62, 10, 30, 50, 86};  
    System.out.print("Original array: ");  
    printArray(data);  
    long startTime = System.nanoTime();  
    quickSort(data, 0, data.length - 1);  
    long endTime = System.nanoTime();  
    System.out.print("Sorted array: ");  
    printArray(data);  
    System.out.println("Execution Time: " + (endTime - startTime) + "  
nanoseconds");  
}
```

➤ **Output: -**

```
PS D:\RISHI\ADIT-sem_5\DAA> javac QuickSort.java  
PS D:\RISHI\ADIT-sem_5\DAA> java QuickSort  
Original array: [34, 7, 23, 32, 5, 62]  
Sorted array:   [5, 7, 23, 32, 34, 62]  
Execution Time: 24200 nanoseconds  
PS D:\RISHI\ADIT-sem_5\DAA> javac QuickSort.java  
PS D:\RISHI\ADIT-sem_5\DAA> java QuickSort  
Original array: [34, 7, 23, 32, 5, 62, 10, 30, 50, 86]  
Sorted array:   [5, 7, 10, 23, 30, 32, 34, 50, 62, 86]  
Execution Time: 7400 nanoseconds  
PS D:\RISHI\ADIT-sem_5\DAA> █
```

➤ **Conclusion: -**

Quick Sort is an efficient sorting algorithm for large datasets. It performs better than bubble, insertion, or selection sort in most practical scenarios. Although its worst case is $O(n^2)$, it's very fast on average due to efficient partitioning.

Practical: - 7

Aim: - Write a program to implement making change problem using greedy algorithm. Write a program to implement making change problem using greedy algorithm.

❖ Make Change Problem Using Greedy Algorithm.

- The Making Change Problem involves determining the least number of coins or currency notes needed to make a given amount of change. This is done using a given set of denominations.
- The Greedy Algorithm approach always picks the largest denomination that does not exceed the remaining amount, continuing this process until the amount becomes zero.
- This approach works optimally when the currency system is canonical (like most modern currency systems such as Indian, US, or Euro denominations).

❖ Greedy Strategy

1. Sort the available denominations in descending order.
2. Iterate through each denomination:
 - Use as many of that denomination as possible.
 - Subtract the total from the amount.
3. Repeat until the amount becomes zero.

Example:-

- Given denominations: ₹1, ₹2, ₹5, ₹10, ₹20, ₹50, ₹100, ₹200, ₹500, ₹2000
- Amount: ₹2758
- Output:
2000 x 1 = 2000, 500 x 1 = 500, 200 x 1 = 200, 50 x 1 = 50, 5 x 1 = 5, 2 x 1 = 2, 1 x 1 = 1

❖ Code:-

```
import java.util.Arrays;
class MakeChangeProblem{
    static int makeChange(int coins [], int amount) {
        // handling the base cases like if coins empty or if amount is 0
        if(coins.length==0){
            System.out.println("No coins available for the change");
            return 0;
        }
        if(amount<=0){
            System.out.println("Amount is 0 or negative, please enter amount in positive");
            return 0;
        }
        Arrays.sort(coins);
        int res=0;
        // loop for calculating coins
        for(int i=coins.length-1;i>=0;i--){
            if(amount>=coins[i]){
                // maximum amount of coins
                int cnt=(amount/coins[i]);

                // increment count of coins used
                res+=cnt;

                // subtracting the coins amount from total amount
            }
        }
    }
}
```

```

        amount-=(cnt*coins[i]);
    }

    if(amount==0)
        break;
    }
    return res;
}

public static void main(String args[]){
    int[][] testCoins = {
        {1, 2, 5, 10},{1, 5, 10, 25},{},{1, 2, 5},{1, 3, 4}
    };
    int[] testAmounts = {39, 25, 10, 0, 6, 289, 18, -34};
    for (int i = 0; i < testCoins.length; i++) {
        System.out.println("Test Case " + (i + 1));
        System.out.println("Coins: " + Arrays.toString(testCoins[i]));
        System.out.println("Amount: " + testAmounts[i]);
        System.out.println("Minimum no of coins used: " +
makeChange(testCoins[i], testAmounts[i]));
        System.out.println("-----");
        System.out.println();
    }
}
}

```

❖ Output: -

```

Test Case 1
Coins: [1, 2, 5, 10]
Amount: 39
Minimum no of coins used: 6
-----

Test Case 2
Coins: [1, 5, 10, 25]
Amount: 25
Minimum no of coins used: 1
-----

Test Case 3
Coins: []
Amount: 10
No coins available for the change
Minimum no of coins used: 0
-----

```

❖ Conclusion: -

The Greedy Algorithm is an efficient solution for the Making Change Problem when the set of denominations allows an optimal greedy solution. In this experiment, we successfully implemented the algorithm to minimize the number of coins/notes used for any input amount.

Practical: - 8

Aim: - Write a program to implement the knapsack problem using greedy algorithm.

➤ **Knapsack Problem using Greedy Algorithm**

- Problem Description:

We are given:

- n items, each with a weight ($w[i]$) and a value ($v[i]$).
- A knapsack with maximum capacity W .
- Goal: Fill the knapsack to maximize the total value, but the total weight must not exceed capacity W .

Note:

- The greedy method works optimally only for the Fractional Knapsack Problem (items can be broken into fractions).
- For the 0/1 Knapsack Problem (items cannot be split), greedy does not always give the optimal solution.

➤ **Algorithm (Fractional Knapsack – Greedy Approach)**

Step 1: Calculate the value/weight ratio for each item.

Step 2: Sort items in descending order of ratio.

Step 3: Initialize total value = 0, remaining capacity = W .

Step 4: Traverse sorted items:

 If item weight $\leq$ remaining capacity $\rightarrow$ take full item.

 Else $\rightarrow$ take fractional part of item to fill the knapsack.

Step 5: Stop when the knapsack is full.

➤ **Code:**

```
import java.util.*;
class Item {
    int value, weight;
    Item(int value, int weight) {
        this.value = value;
        this.weight = weight;
    }
}
public class FractionalKnapsack {

    // Function to get maximum value in knapsack
    public static double getMaxValue(Item[] items, int capacity) {
        // Sort items by value-to-weight ratio (descending)
        Arrays.sort(items, (a, b) -> Double.compare((double) b.value / b.weight,
        (double) a.value / a.weight));

        double totalValue = 0.0;
        int remainingCapacity = capacity;

        for (Item item : items) {
            if (item.weight <= remainingCapacity) {
                // Take whole item
                totalValue += item.value;
                remainingCapacity -= item.weight;
            }
        }
    }
}
```

```
        } else {
            // Take fraction of item
            totalValue += (double) item.value * ((double) remainingCapacity /
item.weight);
            break; // knapsack full
        }
    }
    return totalValue;
}

public static void main(String[] args) {
    // Example items
    Item[] items = {
        new Item(60, 10),
        new Item(100, 20),
        new Item(120, 30)
    };

    int capacity = 50;

    double maxValue = getMaxValue(items, capacity);
    System.out.println("Maximum value in Knapsack = " + maxValue);
}
```

➤ Output:-

```
PS D:\RISHI\ADIT-sem_5\DAA> java FractionalKnapsack
Error: Could not find or load main class FractionalKnapsack
Caused by: java.lang.ClassNotFoundException: FractionalKnapsack
PS D:\RISHI\ADIT-sem_5\DAA> java FractionalKnapsack
Maximum value in Knapsack = 240.0
```

➤ Conclusion:

The Greedy Algorithm is efficient ($O(n \log n)$) and works optimally for Fractional Knapsack Problem. It selects items based on the highest value-to-weight ratio. However, for 0/1 Knapsack, greedy may fail to give the optimal solution, and Dynamic Programming is preferred in that case.

Practical: - 9

Aim: - Write a program to implement making change problem using dynamic programming.

➤ Making Change Problem (Dynamic Programming)

Problem Description

- Given:
 - A set of coin denominations coins[]
 - A target value amount
- Goal:
 - Find the minimum number of coins needed to make the amount.
 - If not possible, return -1.

This is a classic Dynamic Programming problem, often called the Coin Change Problem.

➤ Algorithm (Dynamic Programming – Bottom-Up)

Step 1: Initialize a DP array dp[amount+1], where dp[i] = minimum coins required for value i.

Step 2: Set dp[0] = 0 (0 coins needed for amount 0).

Step 3: Initialize all other dp[i] as ∞ (or a large number).

Step 4: For each coin c in coins:

- For each value i from c to amount:
 - Update dp[i] = min(dp[i], dp[i-c] + 1).
- Step 5: Answer = dp[amount]. If still ∞ , return -1.

Complexity:

- Time Complexity: $O(n * \text{amount})$ where n = number of coins.
- Space Complexity: $O(\text{amount})$

➤ Code:

```
import java.util.*;

public class CoinChangeDP {

    // Function to find minimum coins required
    public static int coinChange(int[] coins, int amount) {
        int MAX = amount + 1; // large number for initialization
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, MAX);
        dp[0] = 0; // base case

        // Bottom-up DP
        for (int coin : coins) {
            for (int i = coin; i <= amount; i++) {
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);
            }
        }

        return dp[amount] == MAX ? -1 : dp[amount];
    }

    public static void main(String[] args) {
        int[] coins = { 1, 3, 4 }; // coin denominations
        int amount = 9; // target amount
    }
}
```

```
int result = coinChange(coins, amount);

if (result == -1) {
    System.out.println("It is not possible to make the given amount.");
} else {
    System.out.println("Minimum coins required = " + result);
}
}
```

➤ Output:-

```
PS D:\RISHI\ADIT-sem_5\DAA> java CoinChangeDP
Minimum coins required = 2
PS D:\RISHI\ADIT-sem_5\DAA> javac CoinChangeDP.java
PS D:\RISHI\ADIT-sem_5\DAA> java CoinChangeDP
Minimum coins required = 3
```

➤ Conclusion:

Dynamic Programming efficiently solves the Making Change Problem in $O(n \times \text{amount})$ time. Unlike greedy algorithms, it always gives the optimal solution, even when coin denominations don't follow the greedy-choice property. This approach can also be extended to count number of ways to make change (variation of problem).

Practical: - 10

Aim: - Write a program to implement the knapsack problem using dynamic programming.

➤ **0/1 Knapsack Problem (Dynamic Programming)**

- Problem Description
 - We are given:
 - n items, each with a weight $w[i]$ and value $v[i]$.
 - A knapsack capacity W .
 - Goal:
 - Maximize the total value of selected items.
 - Constraint: Total weight $\leq W$.
 - Condition:
 - 0/1 Knapsack $\rightarrow$ either take the whole item or leave it (no fractions allowed).

➤ **Algorithm (Dynamic Programming – Bottom-Up)**

Step 1: Create a DP table $dp[n+1][W+1]$, where:

- $dp[i][w]$ = maximum value using first i items with capacity w .

Step 2: Initialization:

- $dp[0][w] = 0$ for all w (0 items = 0 value).
- $dp[i][0] = 0$ for all i (0 capacity = 0 value).

Step 3: Recurrence:

For each item i and capacity w :

if ($weight[i-1] \leq w$)

$dp[i][w] = \max(value[i-1] + dp[i-1][w - weight[i-1]], dp[i-1][w]);$

else

$dp[i][w] = dp[i-1][w];$

Step 4: Answer = $dp[n][W]$.

Time Complexity: $O(n*W)$

Space Complexity: $O(n*W)$ (can be optimized to $O(W)$)

➤ **Code:**

```
public class KnapsackDP {  
  
    // Function to solve 0/1 Knapsack problem  
    public static int knapSack(int W, int[] wt, int[] val, int n) {  
        int[][] dp = new int[n + 1][W + 1];  
  
        // Build table dp[][] in bottom-up manner  
        for (int i = 0; i <= n; i++) {  
            for (int w = 0; w <= W; w++) {  
                if (i == 0 || w == 0)  
                    dp[i][w] = 0;  
                else if (wt[i - 1] <= w)  
                    dp[i][w] = Math.max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);  
                else  
                    dp[i][w] = dp[i - 1][w];  
            }  
        }  
    }  
}
```

```
    }  
}  
  
    return dp[n][W]; // maximum value for full capacity  
}  
  
public static void main(String[] args) {  
    int[] val = { 60, 100, 120 }; // values  
    int[] wt = { 10, 20, 30 }; // weights  
    int W = 50; // knapsack capacity  
    int n = val.length;  
  
    int maxValue = knapSack(W, wt, val, n);  
    System.out.println("Maximum value in Knapsack = " + maxValue);  
}  
}
```

➤ **Output:-**

```
PS D:\RISHI\ADIT-sem_5\DAA> javac KnapsackDP.java  
PS D:\RISHI\ADIT-sem_5\DAA> java KnapsackDP  
Maximum value in Knapsack = 220  
PS D:\RISHI\ADIT-sem_5\DAA> █
```

➤ **Conclusion:**

Dynamic Programming provides an optimal solution to the 0/1 Knapsack problem in $O(n*W)$ time. Unlike Greedy (which only works for Fractional Knapsack), DP ensures the correct solution for 0/1 constraints. The approach can be optimized further for space ($O(W)$).

Practical: - 11

Aim: - Write a program to implement Floyd's algorithm for finding shortest path using dynamic programming.

➤ **Floyd's Algorithm (Floyd–Warshall Algorithm)**

- **Problem Description**

- Input:
 - A weighted graph represented as an adjacency matrix.
 - $\text{graph}[i][j]$ = weight of edge from $i \rightarrow j$ (∞ if no edge).
- Output:
 - Shortest path distances between all pairs of vertices.

➤ **Algorithm (Dynamic Programming)**

Step 1: Initialize $\text{dist}[][] = \text{graph}[][]$.

Step 2: For each vertex k (intermediate):

For each vertex i :

For each vertex j :

if $\text{dist}[i][k] + \text{dist}[k][j] < \text{dist}[i][j]$

$\text{dist}[i][j] = \text{dist}[i][k] + \text{dist}[k][j]$;

Step 3: After processing all vertices, $\text{dist}[i][j]$ contains shortest distance from $i \rightarrow j$.

Time Complexity: $O(n^3)$

Space Complexity: $O(n^2)$

➤ **Code:**

```
public class FloydWarshall {

    final static int INF = 99999; // Infinity value

    // Function to implement Floyd-Warshall Algorithm
    public static void floydWarshall(int graph[][], int V) {
        int dist[][] = new int[V][V];

        // Initialize distance with input graph matrix
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                dist[i][j] = graph[i][j];
            }
        }

        // Floyd-Warshall algorithm
        for (int k = 0; k < V; k++) {
            for (int i = 0; i < V; i++) {
                for (int j = 0; j < V; j++) {
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
            }
        }

        // Print shortest distance matrix
        printSolution(dist, V);
    }
}
```

```
}

// Utility function to print the matrix
public static void printSolution(int dist[][], int V) {
    System.out.println("Shortest distances between every pair of vertices:");
    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                System.out.print("INF ");
            else
                System.out.print(dist[i][j] + " ");
        }
        System.out.println();
    }
}

// Main method
public static void main(String[] args) {
    int V = 4;
    int graph[][] = {
        { 0, 5, INF, 10 },
        { INF, 0, 3, INF },
        { INF, INF, 0, 1 },
        { INF, INF, INF, 0 }
    };

    floydWarshall(graph, V);
}
}
```

➤ **Output:-**

```
PS D:\RISHI\ADIT-sem_5\DAA> javac FloydWarshall.java
PS D:\RISHI\ADIT-sem_5\DAA> java FloydWardshall
Error: Could not find or load main class FloydWardshall
Caused by: java.lang.ClassNotFoundException: FloydWardshall
PS D:\RISHI\ADIT-sem_5\DAA> java FloydWarshall
Shortest distances between every pair of vertices:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0
PS D:\RISHI\ADIT-sem_5\DAA> █
```

➤ **Conclusion:**

Floyd's Algorithm is a Dynamic Programming approach to compute shortest paths between all pairs of vertices. It works for both directed and undirected weighted graphs (with positive or negative edges, but no negative cycles). Complexity is $O(n^3)$, making it suitable for dense graphs or when we need all-pairs shortest paths.

Practical: - 12

Aim: - Write a program to implement chained matrix multiplication using dynamic programming.

➤ **Matrix Chain Multiplication (Chained Matrix Multiplication)**

- **Problem Description**

- We are given a sequence of matrices $A_1, A_2, \dots, A_n$.
- Matrix A_i has dimensions $p[i-1] \times p[i]$.
- Goal: **Find the minimum number of scalar multiplications** needed to multiply the chain.
- Order of multiplication matters because matrix multiplication is associative but not commutative.

➤ **Algorithm (Dynamic Programming – Bottom-Up)**

Step 1: Define $m[i][j]$ = minimum number of multiplications needed to compute product of matrices from i to j .

Step 2: Initialization:

- $m[i][i] = 0$ for all i (single matrix $\rightarrow$ no cost).

Step 3: For chain length $L = 2$ to n :

- For each starting index i :
 - Set $j = i + L - 1$
 - Compute:
 - $m[i][j] = \min (m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j])$ for all $k (i \leq k < j)$

Step 4: Answer = $m[1][n-1]$.

Time Complexity: $O(n^3)$

Space Complexity: $O(n^2)$

➤ **Code:**

```
public class MatrixChainMultiplication {

    // Function to find minimum multiplication cost
    public static int matrixChainOrder(int p[], int n) {
        int[][] m = new int[n][n]; // DP table

        // cost of one matrix is zero
        for (int i = 1; i < n; i++) {
            m[i][i] = 0;
        }

        // L = chain length
        for (int L = 2; L < n; L++) {
            for (int i = 1; i < n - L + 1; i++) {
                int j = i + L - 1;
                m[i][j] = Integer.MAX_VALUE;

                for (int k = i; k < j; k++) {
                    int cost = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
                    if (cost < m[i][j]) {
                        m[i][j] = cost;
                    }
                }
            }
        }
    }
}
```

```
        return m[1][n - 1]; // minimum cost for full chain
    }

    public static void main(String[] args) {
        // Example: A1(10x30), A2(30x5), A3(5x60)
        int arr[] = { 10, 30, 5, 60 }; // dimensions
        int n = arr.length;

        System.out.println("Minimum number of multiplications is " +
            matrixChainOrder(arr, n));
    }
}
```

➤ Output:-

```
PS D:\RISHI\ADIT-sem_5\DAA> javac MatrixChainMultiplication.java
PS D:\RISHI\ADIT-sem_5\DAA> java MatrixChainMultiplication
Minimum number of multiplications is 4500
PS D:\RISHI\ADIT-sem_5\DAA> █
```

➤ Conclusion:

Chained Matrix Multiplication is a classic DP problem. It ensures the optimal parenthesization of matrices to minimize computation cost. Time complexity is $O(n^3)$, but much better than trying all parenthesizations ($O(2^n)$). Useful in compiler optimization and large-scale matrix computations.

Practical: - 13

Aim: - Write a program to implement longest common subsequence using dynamic programming.

➤ Longest Common Subsequence (LCS)

- Problem Description

- Given two sequences (strings), find the **length of their longest subsequence** that appears in both.
- Subsequence = sequence that appears in the same order but not necessarily contiguous.

Example:

X = "AGGTAB", Y = "GXTXAYB"

LCS = "GTAB" → length = 4

➤ Algorithm (Dynamic Programming – Bottom-Up)

Step 1: Let $X[0..m-1]$ and $Y[0..n-1]$ be the two sequences.

Define $dp[i][j]$ = length of LCS of $X[0..i-1]$ and $Y[0..j-1]$.

Step 2: Base case:

- $dp[0][j] = 0$ for all j
- $dp[i][0] = 0$ for all i

Step 3: Recurrence:

if $X[i-1] == Y[j-1]$:

$dp[i][j] = 1 + dp[i-1][j-1]$

else:

$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

Step 4: Answer = $dp[m][n]$

Time Complexity: $O(m*n)$

Space Complexity: $O(m*n)$

➤ Code:

```
public class LongestCommonSubsequence {
    // Function to find length of LCS
    public static int lcs(String X, String Y) {
        int m = X.length();
        int n = Y.length();
        int[][] dp = new int[m + 1][n + 1];
        // Build table in bottom-up manner
        for (int i = 0; i <= m; i++) {
            for (int j = 0; j <= n; j++) {
                if (i == 0 || j == 0)
                    dp[i][j] = 0;
                else if (X.charAt(i - 1) == Y.charAt(j - 1))
                    dp[i][j] = 1 + dp[i - 1][j - 1];
                else
                    dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
        return dp[m][n];
    }
    // Function to reconstruct the LCS string
    public static String getLCS(String X, String Y) {
        int m = X.length();
        int n = Y.length();
        int[][] dp = new int[m + 1][n + 1];
```

```
// Fill DP table
for (int i = 0; i <= m; i++) {
    for (int j = 0; j <= n; j++) {
        if (i == 0 || j == 0)
            dp[i][j] = 0;
        else if (X.charAt(i - 1) == Y.charAt(j - 1))
            dp[i][j] = 1 + dp[i - 1][j - 1];
        else
            dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
    }
}
// Reconstruct the LCS string
int i = m, j = n;
StringBuilder lcsStr = new StringBuilder();
while (i > 0 && j > 0) {
    if (X.charAt(i - 1) == Y.charAt(j - 1)) {
        lcsStr.append(X.charAt(i - 1));
        i--;
        j--;
    } else if (dp[i - 1][j] > dp[i][j - 1]) {
        i--;
    } else {
        j--;
    }
}
return lcsStr.reverse().toString();
}
public static void main(String[] args) {
    String X = "AGGTAB";
    String Y = "GXTXAYB";
    int length = lcs(X, Y);
    String sequence = getLCS(X, Y);
    System.out.println("Length of LCS = " + length);
    System.out.println("LCS sequence = " + sequence);
}
}
```

➤ **Output:-**

```
PS D:\RISHI\ADIT-sem_5\DAA> javac LongestCommonSubsequence.java
PS D:\RISHI\ADIT-sem_5\DAA> java LongestCommonSubsequence
Length of LCS = 4
LCS sequence = GTAB
```

➤ **Conclusion:**

LCS is solved efficiently using Dynamic Programming with $O(m*n)$ time. It has important applications in: Bioinformatics (DNA/protein sequence matching). Diff tools (file comparison, version control like Git). Spell checking and plagiarism detection. Both the length and the subsequence can be extracted from the DP table.